AI Tools for Software Bug Detection

Detector timing and fixture quality checks

On twelve Python functions from the Task 2 repository, median scoring time was **0.1073 ms for AST rules** and **0.0151 ms for Naive Bayes**. Both methods received the same source text. The functions have no independent defect labels, so this comparison answers the runtime question only.

Performance on repository code

Per-method measure	AST rules	Naive Bayes
Median latency, ms (p25–p75)	0.1073 (0.0580–0.1726)	0.0151 (0.0109–0.0245)
95% CI for median, ms	0.1000–0.1081	0.0149–0.0154
p95 / p99 latency, ms	1.2150 / 1.2916	0.1513 / 0.1603
Throughput, snippets/s	5,017	35,866
95% CI for throughput, snippets/s	4,856–5,162	34,237–37,108
Failed / measured calls	0 / 360	0 / 360

The interquartile range (p25–p75) describes variation among calls. The confidence intervals use 10,000 bootstrap samples of entire passes, preserving the mix of functions. Throughput divides completed calls by the total time spent scoring.

What was measured

The workload contains six functions from the detector and six from its original tests. Selection used six size ranks per file, including the shortest and longest. Source texts span 73–6,244 UTF-8 bytes. Each scorer processed all twelve functions plus eight labelled fixtures in randomized order.

The collector logged 200 calls during five warm-up passes and 1,200 calls during thirty benchmark passes. Of the measured calls, 720 used repository functions and 480 used fixtures. Seed 42 controlled the input and method order. File I/O, fitting and reporting took place outside each scoring interval.

Run environment

Debian Bookworm · Linux/arm64 · CPython 3.12.10 · Apple M3 host. Docker enforced a one-CPU quota and 512 MiB of memory. All calls ran in one process. Model fitting took 0.089 ms. The collector reached 20.83 MiB peak resident memory through the last scored call.

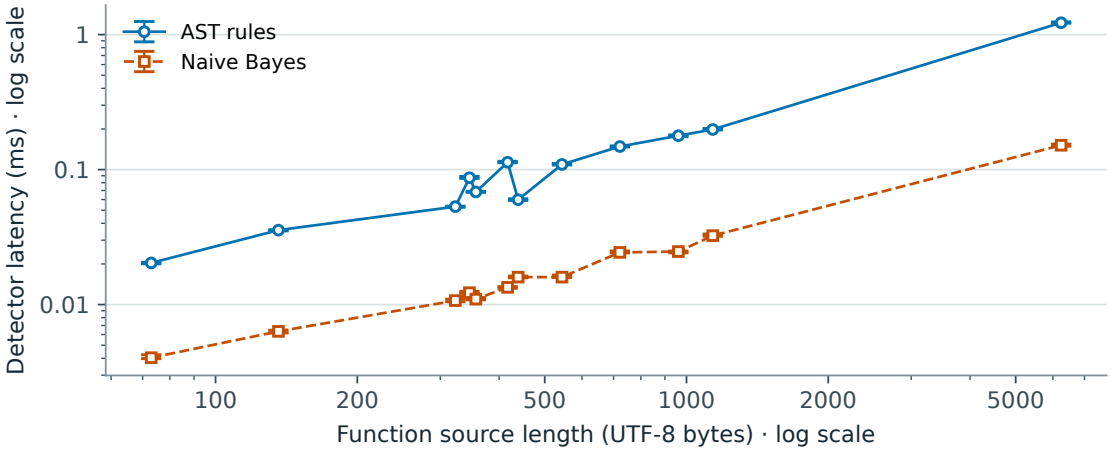
Variation across inputs and calls

Every selected function had a lower median with the lexical classifier. Functions of similar length could still take different times with AST rules. The lines connect distinct functions ordered by source length.

FIGURE 01 / AI TOOLS FOR SOFTWARE BUG DETECTION

Detector time by source length

12 unlabelled repository functions · 30 benchmark passes per detector

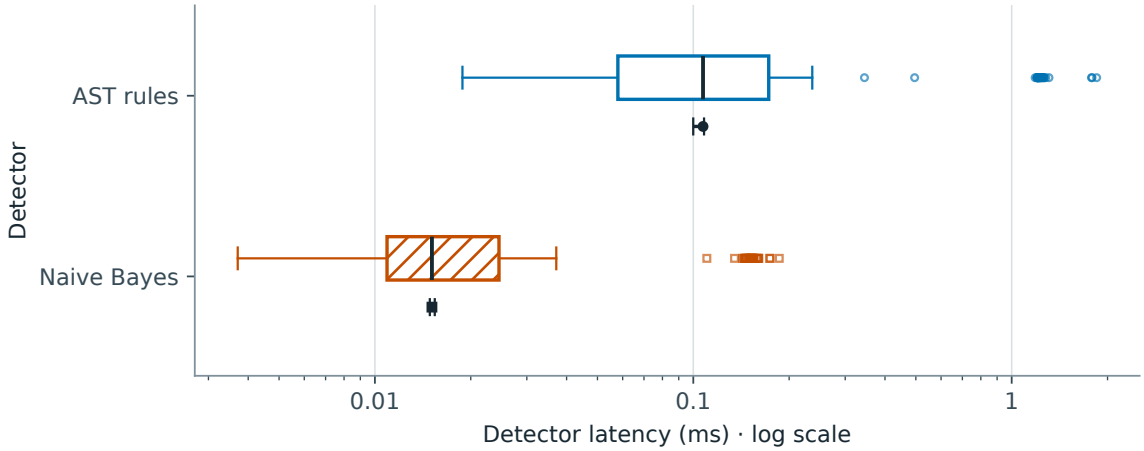


Points are function medians. Caps show pointwise 95% intervals from resampling passes (10,000 draws). Some intervals are narrower than the markers. Lines connect functions in size order.

FIGURE 02 / AI TOOLS FOR SOFTWARE BUG DETECTION

Spread of individual detector times

720 successful calls · 12 unlabelled repository functions · 30 benchmark passes



Boxes span the middle 50%. Whiskers reach the last observations within 1.5 IQRs. Open marks show tail calls. Below each box, the capped mark gives the median and its 95% interval from resampling whole passes.

The boxes include all successful benchmark calls. Flagging extreme times within a function and detector removed no observations. Variation includes input composition and scheduling on a single virtualized host.

What the fixture labels establish

The trained model uses the vocabulary of eight synthetic training fixtures, empirical class priors and add-one smoothing. A posterior score of at least 0.5 flags a snippet. The fixed rules parse the Python AST to identify literal zero divisors and mutable literal defaults.

Eight labelled test fixtures

One prediction per snippet	AST rules	Naive Bayes
TP / FP / TN / FN	4 / 0 / 4 / 0	4 / 2 / 2 / 0
Precision / recall / F1	1.000 / 1.000 / 1.000	0.667 / 1.000 / 0.800
False-positive rate (four negatives)	0.00	0.50
F1 for division by zero (four cases)	1.000	0.667
F1 for mutable defaults (four cases)	1.000	1.000

Naive Bayes flagged both negative division-by-zero fixtures. The paired F1 difference (NB minus AST) was −0.200. The 95% bootstrap interval across four synthetic project groups was [−0.333, 0.000]. All 10,000 paired draws contained both classes. Scoring the same snippet repeatedly adds timing information but supplies only one labelled case.

Registered decision and limits

Task 2 specifies at least 50 independently reviewed training snippets and 100 reviewed test snippets from at least 10 held-out projects. Both authors must label the examples and reconcile disagreements before freezing the data. Those labels are unavailable. The question of an F1 improvement on real software remains open.

A practical success requires a positive lower 95% bound for the paired F1 difference, an observed gain of at least 0.05 and all guardrails. The p95 limit of 100 ms held for both methods on both workloads. All 1,400 calls succeeded. The shared collector peak was below 256 MiB. NB exceeded the 0.10 FPR limit on fixtures. Real-code FPR and F1 cannot be calculated from this log.

These resource observations cover one run of one repository in a container. The guest advertised 1 ms clock resolution. Among 50,000 adjacent reads the minimum positive increment was 41 ns. The RSS observation includes training and buffered events, and precedes JSONL serialization and the separate analysis process.

Provenance and replay

Project revision **374dea0**. The log header lists every input, line range, byte count and source hash. The notebook rebuilds the CSVs and both figures from the frozen log. The saved JSON model reproduced all 700 NB scores and predictions.

Recorded run 9eaaf526-e5b4-4c8a-ba28-ae2c1bd9ecf3
Raw log SHA-256 ef7a7fb5a6208d53768159fba07af7f2e5d20891e969d851bc27ac46b18a4685

Open the frozen Task 2 project on GitHub